NGO Donation Transparency Tracker

A blockchain-based donation and expenditure tracking system for NGOs

Project explanation in simple words

Mini Project · Blockchain (CSDC7022) · Semester VII

The whole idea in one line: a website where NGOs collect donations, and every rupee that comes in and every rupee that goes out is written on a blockchain, so anyone can check it and nobody can secretly change it.

Think of a glass donation box in the middle of a market. Everyone can see how much money went in. The NGO cannot take money out without a matching bill: an automatic checker inspects every withdrawal first. Every time money comes out, a note is stuck on the glass: “₹500 taken out for rice bags, paid to Sharma Traders, bill attached.” Those notes can never be removed or edited. In this project the glass box is a **smart contract**, and the notes are **blockchain records**.

1. Why this project is needed

- India has about **33 lakh registered NGOs** (roughly one for every 400 people), but **fewer than 10%** file their yearly financial returns, and only a small number hold an FCRA licence. Most work with very little independent audit.
- Bain & Company's India Philanthropy Report found that **50% of donors could not name an NGO they trusted** enough to donate to.
- Today, donors learn how their money was spent from a **PDF report written by the NGO itself**, often months later. There is no way to check it.
- Small and new NGOs, and sudden disaster-relief collections (floods, cyclones), suffer most, because nobody knows them yet.

> **Our answer:** do not ask donors to trust the NGO. Give them a way to check every rupee themselves.

2. A few words first

Word	Simple meaning
Blockchain	A shared notebook copied on many computers. Once something is written in it, it cannot be erased or changed.
Smart contract	A small program living on the blockchain that follows fixed rules automatically (e.g. “no money is released without a receipt that has never been used before”). Nobody can bend the rules, not even its author.
Wallet / address	Your account on the blockchain, like a bank account number (e.g. 0x7099...79C8). In this project your address **is** your identity; there are no usernames or passwords.
Private key	The secret password of a wallet. Whoever has it can use the wallet. MetaMask keeps it for you.
ETH	The currency of Ethereum. In this project it is **test ETH** on a practice blockchain: it has no real value.
Transaction	Any action that changes something on the blockchain (donate, create a cause, spend funds). It is recorded permanently in a block and costs a small fee called **gas**.
Block	A page of the notebook. Each new block holds the latest transactions; on our local chain, every transaction gets its own block.
Event	A message the smart contract writes into the blockchain log when something happens (e.g. “DonationReceived”). The Activity tab is built from these.

3. The 20 test accounts (addresses)

When you run **npm run node**, Hardhat starts a private practice blockchain on your own laptop and prints 20 accounts, each with an address and a private key. This section explains what they are.

Where do they come from, and who created them?

- They are created **automatically by Hardhat** (the developer tool we use), not by us and not by any bank.
- Hardhat generates them from a fixed, publicly known test phrase (`test test test test test test test test test test test junk`). Because the phrase is always the same, **you get the same 20 addresses and private keys every time**, on every computer.
- Their private keys are printed on screen and are **public knowledge** (every Hardhat user has the same ones). That is fine for practice, and exactly why they must **never** be used on a real network.

How much money is in each, and is it real?

- Each account starts with **10,000 ETH**.
- This is **fake test ETH**. It exists only on your local Hardhat blockchain (network "Hardhat Local", chain ID **31337**). It cannot be sent to a real exchange, cannot be converted to rupees, and has **zero real value**.
- MetaMask may show a dollar value (e.g. "$25,088,105"). That is just MetaMask multiplying by the real ETH price; it is **not real money**.
- Every transaction costs a tiny **gas fee** in test ETH. That is why an account shows 9,999.994 ETH after a few actions instead of exactly 10,000.
- The whole thing is a **simulation**: it behaves exactly like the real Ethereum network, but on your laptop only.
- When you stop and restart **npm run node**, the blockchain starts again from zero: every account is back to 10,000 ETH and all causes and donations disappear. That is why you redeploy after a restart.

How we use them: one account per role

In real life each role would be a different person with their own wallet. In our demo we give each role one of the 20 accounts, and you switch between them in MetaMask to play each person.

Account	Address	Role in the demo	Import into MetaMask?
#0	`0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266`	**Owner / platform admin** (Admin tab)	Yes
#1	`0x70997970C51812dc3A010C7d01b50e0d17dc79C8`	**NGO**: creates causes, spends funds (automatically audited)	Yes
#3	`0x90F79bf6EB2c4f870365E785982E1f101E93b906`	**Donor**: donates, claims refunds	Yes
#4	`0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65`	**Supplier 1** (Sharma Traders, receipt1.pdf): receives payments	Optional (to watch its balance)
#5	`0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc`	**Supplier 2** (Gupta Tent House, receipt2.pdf): receives payments	Optional
#2, #6 – #18	`(see terminal)`	Spare: extra donors or NGOs if you want more people in the demo	Optional

Account	Address	Role in the demo	Import into MetaMask?
#19	`0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199`	**Deployer**: only used by the deploy script to put the contract on the chain	**No, never**

> **Why does account #19 deploy the contract?** MetaMask keeps its own count of each account's transactions (the “nonce”). If an account you use in MetaMask also sent transactions from a script, MetaMask's count would fall behind and you would get the error “nonce has already been used”. So the deploy script uses #19, which you never import, and simply names #0 as the owner.

4. MetaMask: what it is and why we use it

- **What:** a free browser extension (add-on for Chrome) that works as your **digital wallet**.
- **Identity / login:** it tells the website which address you are. The website shows you the right buttons for your role (owner, NGO or donor). No username or password is needed.
- **Signing:** every action that changes the blockchain opens a MetaMask pop-up asking you to **Confirm**. Only after you confirm is the transaction sent. This is your digital signature.
- **Holding money:** it holds your (test) ETH and sends it when you donate, and pays the small gas fee.
- **Setup:** add the network “Hardhat Local” (RPC http://127.0.0.1:8545, chain ID 31337, symbol ETH), then **import** the test accounts using their private keys.
- **Viewing needs no MetaMask:** anyone can open the site and read every cause, donation and expenditure.

5. The roles: who does what

Role	Who	Can do	Rules the contract enforces
Owner (platform admin)	Account #0, fixed when the contract is deployed	• Verify cause creators as a **registered organisation** or an **individual** (personal cause, e.g. medical)	Only the owner can do these. The owner **cannot create causes** and **never touches donation money**.
NGO	Whoever creates a cause (demo: #1)	• Create causes • **Spend** the money: amount, purpose, supplier wallet and a **compulsory receipt PDF**	Every payment is audited automatically: receipt required and never reused, amount and supplier must match the receipt, cannot pay itself, cannot spend before the goal or more than remains.
Donor	Anyone (demo: #3)	• Donate • Track their donation • Claim a **refund** if the cause fails	Can only refund their own donation, only once, only if the goal was missed.
Recipient (shop / supplier)	Address typed by the NGO (demo: #4)	Receives payment directly from the contract when the automatic audit passes	Does nothing on the website; just receives ETH.
Public viewer	Anyone, even without a wallet	See every cause, donation, expenditure, receipt and the Activity log	Read-only; costs nothing.

6. Step by step: what happens and where the money goes

Every step below that changes something is a **transaction**: MetaMask asks you to confirm, a small gas fee is paid in test ETH, and the action is written into a new block forever.

Step	Who (where)	What happens	Where the money goes
0. Setup (deploy)	Deployer #19 (script)	The smart contract is put on the local blockchain. Account #0 is set as owner.	No donation money moves. #19 pays the gas.
1. Verify creator	Owner #0 (Admin tab)	After checking documents outside the system, the owner enters the creator's address and chooses **Registered organisation** or **Individual** (e.g. a personal medical fundraiser). Its causes now show the matching badge. (Unverified creators can still create causes.)	No donation money moves.
2. Create a cause	NGO #1 (Create Cause tab)	NGO fills title, description, category, goal and deadline. The NGO's address becomes the only one that can spend this cause's money.	No donation money moves.
3. Donate	Donor #3 (Causes tab)	Donor enters an amount and confirms in MetaMask. The donation (who, how much, when) is recorded forever. The progress bar and totals update for everyone.	Money goes **from the donor's wallet into the smart contract**, held for that cause. It does **not** go to the NGO.
4. Donations close	Automatic	As soon as the total reaches or passes the goal, donations close. The last donation is accepted in full even if it goes over. Donations also close at the deadline.	Money stays in the contract.
5. Spend with automatic audit	NGO #1 (Causes tab)	Only after the goal is reached. NGO enters purpose (“50 rice bags”), amount and supplier wallet, and attaches the **receipt PDF** (compulsory). The website reads the PDF and shows a checklist: receipt attached, never used before, amount and wallet on the receipt match, not paying itself, enough funds. **Audit & Pay** unlocks only when every check is green; the contract re-checks its rules before paying.	If all checks pass: money goes **directly to the supplier**, never through the NGO. If any check fails: **nothing is paid**.
6. Reused receipt	Automatic	Using the same receipt again is blocked: the receipt's fingerprint is marked as used on the blockchain.	No money moves.
7. Verify a receipt	Anyone	Pick the bill file; the website computes its fingerprint and compares it with the one on the blockchain: “Receipt verified” or “does NOT match”.	Not a transaction: free, no money moves.
8. Refund	Donor #3	If the deadline passed and the goal was **not** reached, a Claim refund button appears. Each donor can claim once.	Money goes **from the contract back to the donor's wallet**, exactly the amount they gave.

Step	Who (where)	What happens	Where the money goes
9. All spent	Automatic	When all the money has been paid out, the cause shows **Fully spent · completed** and no more spending is possible.	Contract balance for this cause is 0.

Money flow in one picture

Donor's wallet → donate → **Smart contract** (holds money per cause) → audit passes → **Recipient / shop**

↓ goal missed by deadline
→ refund back to the donor

> **Key point:** the NGO and the owner **never hold the donated money**. It sits in the smart contract and can only leave in two ways: to a supplier after the automatic audit passes, or back to the donor as a refund. The only ETH these roles spend is the tiny gas fee for their own transactions.

7. What is a transaction and what is free?

Transaction (MetaMask pop-up, gas fee, recorded in a block)	Free read (no wallet needed, nothing recorded)
Deploying the contract · Verifying a creator · Creating a cause · Donating · Spending funds (audited payment) · Claiming a refund	Viewing causes, totals and progress · Viewing donation and expenditure lists · Viewing the Activity tab · Verifying a receipt file and the audit checklist (done in the browser)

8. The website (tabs)

Tab	What you find there	Who uses it
Causes	Every cause as a card: category, status (Fundraising / Goal reached · donations closed / Goal missed · refunds open / Fully spent), verification badge (organisation / individual), goal, raised, spent, remaining, progress bar. Donate box, refund button, the NGO's spending form with the automatic-audit checklist, and the full list of donations and expenditures with receipt checks.	Everyone
Create Cause	Form to create a new cause, with the spending rules shown.	NGOs and individuals
Activity	Live table of every on-chain event, newest first, one row each: event, cause, from, to, amount, details, time and block number (causes created, donations, audited payments, refunds, verifications).	Everyone
Admin	Verify creators as a registered organisation or an individual. Only visible to the owner (#0).	Owner only

9. Technology used and what for

Tool	What it is	What we use it for
Solidity	Programming language for smart contracts	Writing the contract: causes, donations, audited spending, refunds, verification
Hardhat	Ethereum developer toolkit	Runs the **local practice blockchain** with 20 test accounts; compiles the contract; runs **26 automated tests**; deploys the contract
Hardhat Network	Private blockchain on your laptop (chain ID 31337)	A free, fast, safe place to demo everything with fake ETH
MetaMask	Browser wallet extension	Identity (who you are), confirming/signing transactions, holding test ETH
ethers.js	JavaScript library	Connects the website to the blockchain: reads data and sends transactions
HTML, CSS, JavaScript	Basic web languages	The website: tabs, cards, forms and buttons
Node.js + npm	JavaScript runtime and package installer	Runs Hardhat; installs everything with one command (npm install)
http-server	Tiny web server	Serves the website at http://127.0.0.1:8080
pdf.js	PDF reader library (Mozilla)	Reads the receipt PDF in the browser so its amount and supplier wallet can be checked automatically
SHA-256	Hashing method (file fingerprint)	Turning a receipt file into a fingerprint stored on-chain, so any later change is detectable
IPFS (optional)	Decentralised file storage	Optional place to keep the actual receipt file, linked from the blockchain
Git + GitHub	Version control	Saving every step of the project's history

Note: the original proposal listed Remix IDE and Ganache. Hardhat replaced both, because it provides the local blockchain (like Ganache) and automated testing (like Remix) in one tool.

10. Safety rules built into the smart contract

- **Automatic audit on every payment:** receipt required, amount and supplier must match it, and the contract rejects anything else.
- **Single-use receipts:** a receipt's fingerprint is marked as used, so one bill cannot justify two payments.
- **No self-payment:** the NGO cannot pay itself or the platform owner; money goes straight to the supplier.
- **Spending only after the goal:** so a failed cause still holds all its money and every refund is always payable.
- **Donations close at the goal** (or the deadline).
- **Balance check:** the NGO can never spend more than the cause still holds.
- **Safe refunds (pull payment):** the donor's balance is set to zero before the money is sent, so nobody can trick the contract into paying twice.
- **Tamper-evident receipts:** the bill's fingerprint is on-chain; a swapped or edited bill no longer matches.
- **Everything is public and permanent:** who donated, what was paid, to whom, with which receipt, and when.

11. Demo receipts: how the automatic audit is shown

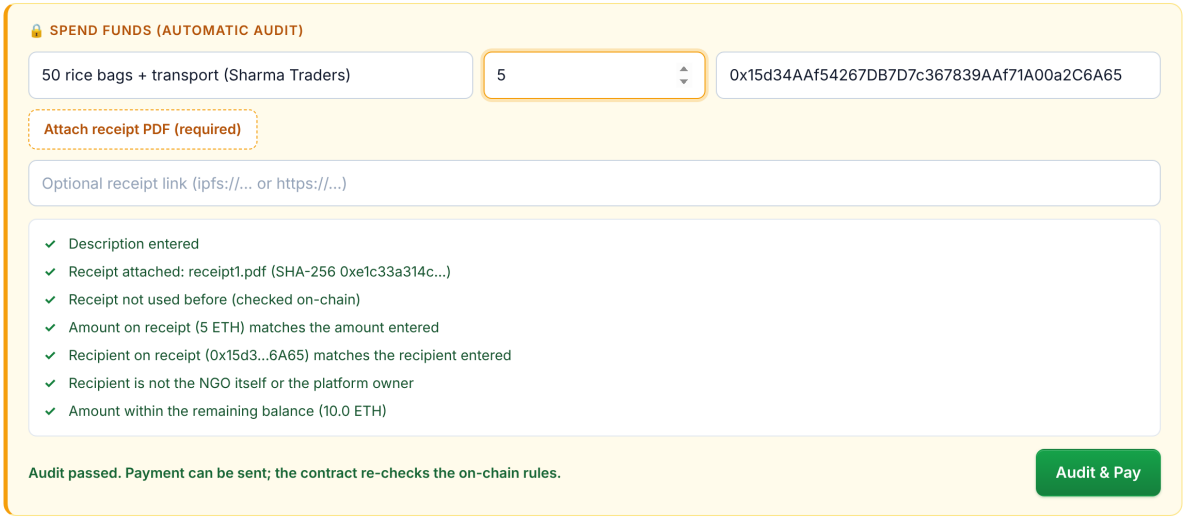
Three receipt PDFs are included in **docs/demo-receipts/** for the demonstration. Each looks like a normal shop bill and contains the two lines the website reads: **Total Amount: … ETH** and **Pay to wallet: 0x…**.

Receipt	From / for	Amount and pay-to wallet	Result
receipt1.pdf	Sharma Traders · 50 rice bags + transport	5 ETH → Supplier1 (account #4, 0x15d3…6A65)	**Passes** · 5 ETH paid to Supplier1
receipt2.pdf	Gupta Tent House · 40 tarpaulins + bamboo poles	3 ETH → Supplier2 (account #5, 0x9965…A4dc)	**Passes** · 3 ETH paid to Supplier2
receipt3.pdf (fake)	Tampered copy of receipt1	5 ETH → **the NGO's own wallet** (account #1, 0x7099…79C8)	**Rejected** · nothing is paid

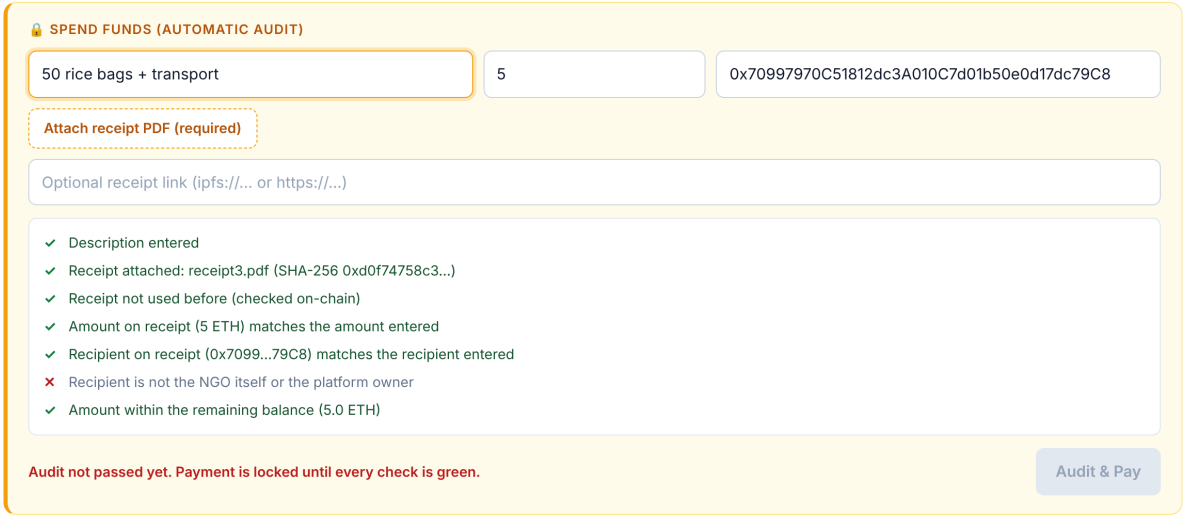
Why receipt3 fails

Someone has changed the “Pay to wallet” line on a real bill to the NGO's own address, so the NGO could quietly pay itself. The audit checklist shows **“Recipient is not the NGO or the owner”** in red and the Audit & Pay button stays locked. Even if someone skipped the website and called the contract directly, the contract itself would reject it with “Recipient cannot be the NGO or the platform owner”. Using receipt1 or receipt2 a second time also fails, with “This receipt has already been used”.

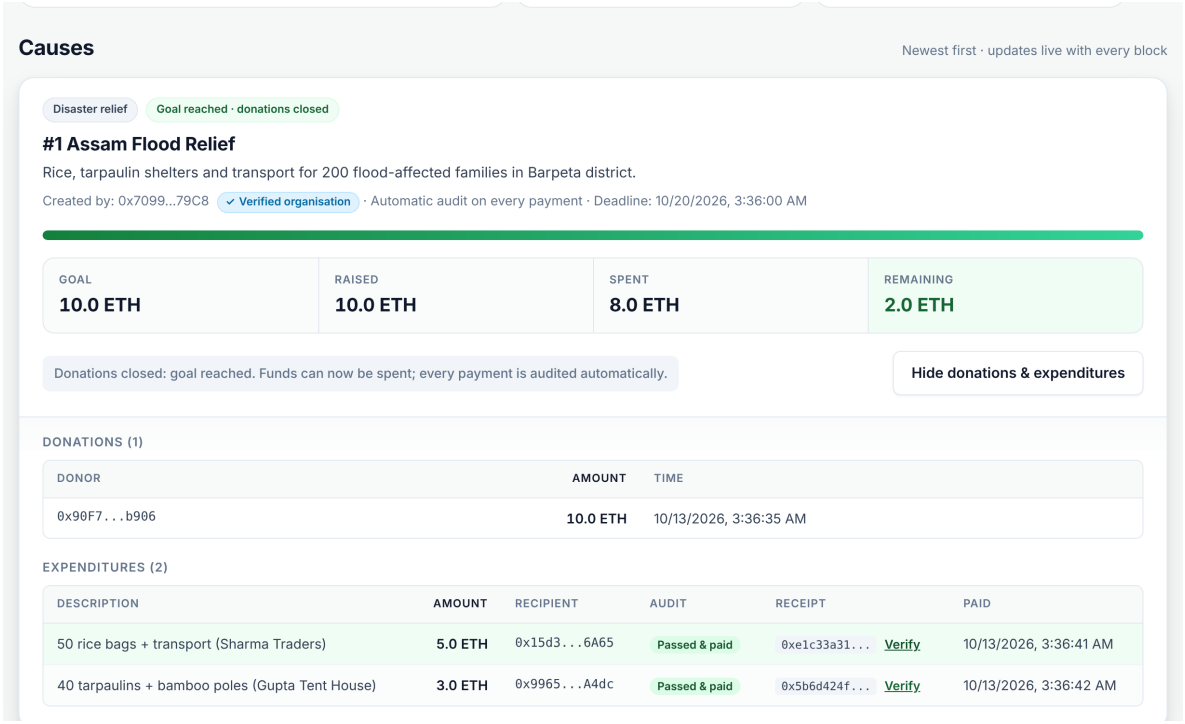
Demo order: donate 10 ETH from the Donor (#3) to reach the goal → as the NGO (#1), attach receipt1 and pay → attach receipt2 and pay → attach receipt3 and show that it is blocked → click **Verify** next to a payment and pick receipt1.pdf to show “Receipt verified”.



receipt1.pdf: every audit check is green, so payment is unlocked.



receipt3.pdf (fake): the self-payment check fails, so payment stays locked.



After both payments: each expenditure shows its recipient, “Passed & paid” and the receipt fingerprint with a Verify link.

12. Conclusion

What the project IS

- A **transparent donation platform** where every rupee in and out is recorded on a blockchain: public, permanent, and not editable by anyone, including the NGO.
- It replaces “trust the NGO's PDF report” with “check it yourself, any time”.
- Real safety rules are written into code: automatic audit with compulsory single-use receipts, spending only after the goal, automatic refunds, receipt fingerprints and verified creators (organisations or individuals).
- A working demonstration of core blockchain ideas: smart contracts, transactions, wallets as identity, immutability, rule-based auditing, hashing and events.

What the project is NOT

- **Not live with real money.** It runs on a local practice blockchain with fake test ETH (10,000 per test account, zero real value).
- **Not a guarantee that the real world matches the records.** The automatic audit proves each payment matches an unaltered, never-reused receipt; it cannot see whether the rice was really delivered or whether the bill itself is genuine.
- **Not a replacement for legal registration or official audits.** The verification badges depend on the owner checking documents outside the system.
- **The PDF checks run in the website.** A smart contract cannot read files, so the amount and wallet on the receipt are compared in the browser; the contract still enforces the receipt, no-reuse, no-self-payment and balance rules.
- **Not a full product.** No login system, no rupee/UPI payments, no mobile app, no multi-chain support. It is a focused college mini project.

> **In one line:** our project does not ask donors to trust NGOs. It lets them verify every rupee themselves.